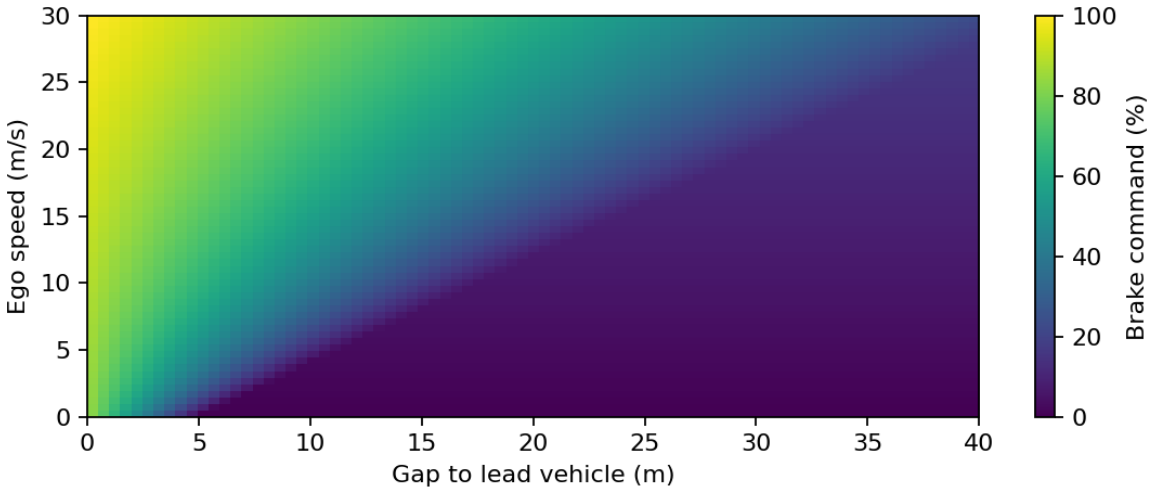
FUZZY LOGIC-BASED INTELLIGENT CAR SPEED CONTROL

Technical Project Report • Python Simulation • Control Logic • Evaluation



Prepared for: Md Mobashwer Hossain
Domain: CSE (Artificial Intelligence & Machine Learning)
Report focus: Smooth deceleration, fuzzy rule design, Python implementation, plots, complexity analysis and visual explanation.

Project boundary: This document demonstrates a software simulation of a vehicle following a lead vehicle. It is not a road-certified braking system and does not claim that camera-based pothole detection has been implemented.

1. Abstract and Project Motivation

This project studies how a controlled vehicle can adjust its longitudinal speed when a vehicle ahead changes the available gap. A Fuzzy Logic Controller (FLC) maps interpretable linguistic inputs—such as Slow/Fast speed and Near/Far gap—to a braking command. A discrete-time vehicle model applies the command, updates speed and position, and logs the response. Python is suitable because it supports numerical modelling, rule evaluation, repeatable experiments and plotting in a compact workflow.

The engineering objective is not to claim universal safety. It is to implement a transparent baseline, test it under documented scenarios, and inspect whether the braking command changes smoothly while speed and gap remain physically meaningful.

Problem statement

A controller based only on hard thresholds may abruptly switch from no braking to strong braking when a measured gap crosses one boundary. Fuzzy sets overlap, allowing partial membership in neighbouring categories. A rate limiter can further limit the rate of change of the command. Both design choices require validation: smooth commands alone do not guarantee safe stopping.

Objectives

• Define measurable inputs, output, units and simulation assumptions.

• Build fuzzy membership functions and a complete IF–THEN rule table.

• Implement gradual deceleration and prevent negative speed.

• Generate speed, gap, braking and control-surface plots.

• Evaluate scenarios with metrics rather than unsupported claims.

• Document limitations and future extensions honestly.

Expected deliverables: reproducible Python source, configuration parameters, plots, scenario logs, evaluation metrics and this report.

2. Scope, Assumptions and System Requirements

Item	Definition used in this report
Controlled object	Ego vehicle (the simulated vehicle whose speed is adjusted).
Lead vehicle	Vehicle in front; its speed and position are available to the simulator.
Inputs	Ego speed v (m/s), bumper-to-bumper gap d (m); relative speed can be added.
Output	Brake command B from 0% (no brake) to 100% (maximum modelled brake).
Time step	$\Delta t = 0.1$ s for the demonstration; smaller steps may be needed for convergence.
Motion model	One-dimensional longitudinal motion; no steering, lane changes or tyre dynamics.
Sensing	Virtual state values; no real camera/radar distance measurement in the baseline.

In scope

• Gap-aware speed control in a repeatable software simulation.

• Fuzzy rule evaluation, smooth command limiting, logging and visual analysis.

• Scenario testing including a lead vehicle slowing down.

Out of scope

• Road deployment, certified emergency braking, steering or lane keeping.

• Direct pothole detection and metric distance estimation from a camera.

• Claims of measured real-world performance without hardware and ground-truth tests.

Requirement checks: all values must be finite; speed and brake command must remain bounded; the lead and ego positions must use consistent units; stopping and gap-violation events must be recorded rather than hidden.

3. Architecture and Feedback Workflow

Module	Input → output	Responsibility
Virtual sensor	State → speed and gap	Provides simulator state; may later add noise/delay.
Fuzzification	Numeric input → membership degrees	Calculates degree of Slow/Medium/Fast and Near/Medium/Far.
Rule inference	Membership degrees → output strengths	Combines IF–THEN rules using min/max or another documented operator.
Defuzzification	Aggregated output → B (%)	Converts fuzzy output to a single command.
Command conditioner	Target B → limited B	Limits change per second and clamps to [0,100].
Vehicle model	B and state → next state	Applies deceleration and updates positions.
Logger/evaluator	Time series → metrics/plots	Records speed, gap, braking, stopping and violation events.

Closed-loop sequence

- 1. Initialise positions, speeds, target speed, time step and scenario duration.
- 2. Measure the current gap and ego speed from the virtual sensor.
- 3. Validate input ranges and compute relative speed if enabled.
- 4. Fuzzify each input and evaluate all relevant rules.
- 5. Aggregate the rule outputs and defuzzify the target brake command.
- 6. Rate-limit and clamp the brake command; apply it to the vehicle model.
- 7. Update ego/lead positions, record metrics and repeat until the run ends.

The loop is closed because the next command depends on the state produced by the previous update. A single one-off calculation is not a complete controller.

4. Mathematical Model and Units

Consistent units are essential: speed in metres per second (m/s), distance in metres (m), time in seconds (s), acceleration in m/s² and brake command in percent.

Gap definition

Let x_e be the ego vehicle's front reference position and x_l be the lead vehicle's rear reference position, using the same longitudinal coordinate convention. Then the bumper-to-bumper gap is d = x_l – x_e. If vehicle reference points are at their centres, vehicle lengths must be included explicitly.

Discrete-time kinematics

At simulation step k, the speed update is:
v[k+1] = max(0, v[k] – a[k] Δt)
and the position update can use:
x[k+1] = x[k] + v[k] Δt
A more accurate constant-acceleration update is x[k+1] = x[k] + v[k]Δt + 0.5 a_signed[k](Δt)².

Brake command to deceleration

For a simple linear actuator model, a[k] = (B[k]/100) a_max, where a[k] is braking deceleration magnitude and a_max is the assumed maximum deceleration. This is a simplified simulation assumption, not a vehicle-specific brake model.

Relative speed and safe-gap proxy

Closing speed c = max(0, v_e – v_l). A simple configurable gap proxy is d_safe = d_0 + T_h v_e, where d_0 is a standstill offset and T_h is a time-headway parameter. This proxy helps define experiments but is not a universal legal or safety threshold.

Bounds: 0 ≤ B ≤ 100; v ≥ 0; Δt > 0. Check units and use a documented sign convention for acceleration.

5. Fuzzy Logic: Core Concepts

Fuzzy logic represents gradual categories. Unlike a binary rule such as “gap below 10 m = brake”, a fuzzy input can belong partly to Near and partly to Medium. This makes a graded response possible around category boundaries.

Term	Meaning	Example
Fuzzification	Map a numeric value to membership degrees.	A 12 m gap may be 0.4 Near and 0.6 Medium.
Membership function	Curve mapping input to degree μ in [0,1].	Triangular or trapezoidal set.
Rule base	Human-readable IF–THEN statements.	IF gap is Near AND speed is Fast THEN brake is High.
Inference	Combine active rules and their strengths.	Minimum for AND; maximum for aggregation.
Defuzzification	Convert fuzzy output to one number.	Centroid or weighted average.
Rate limiting	Restrict how quickly the command changes.	Maximum 22 percentage points per second.

Why use fuzzy logic?

- Rules are interpretable and can encode qualitative control intuition.
- Overlapping sets can reduce abrupt transitions caused by hard boundaries.
- The design can be inspected through a rule table and control-surface plot.

Important caution: fuzzy logic does not automatically guarantee stability, comfort or safety. Membership functions, rules, actuator limits and the vehicle model must all be tested.

6. Membership Functions and Normalisation

A triangular membership function with parameters (a,b,c) can be defined as:

μ(x;a,b,c) = 0, when x ≤ a or x ≥ c; (x–a)/(b–a), when a < x ≤ b; (c–x)/(c–b), when b < x < c. Special endpoint cases must be handled when a=b or b=c.

Illustrative speed sets

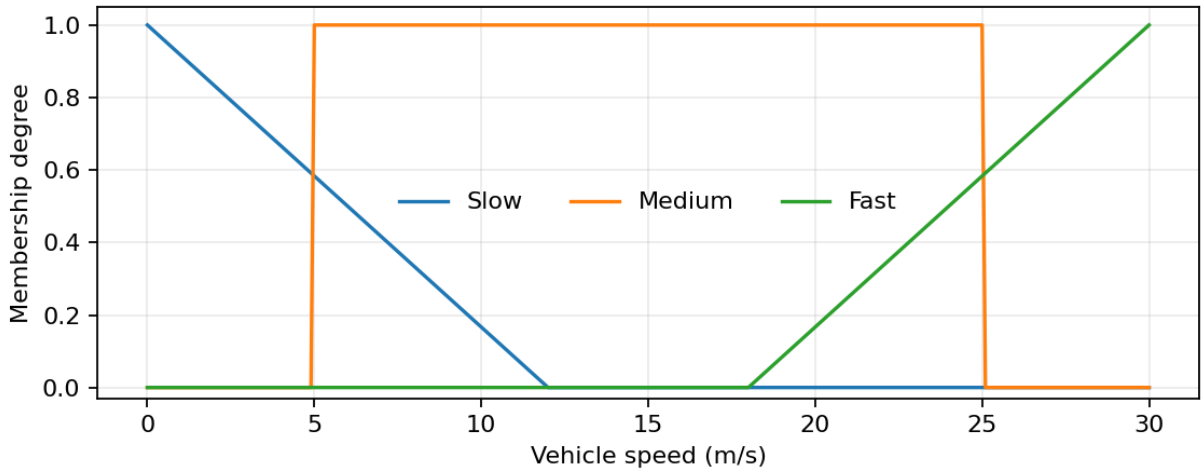


Figure 1. Example overlapping membership functions. The curves are a design illustration; the parameters must be tuned and validated for the chosen operating range.

Variable	Example fuzzy sets	Illustrative ranges
Speed v	Slow, Medium, Fast	0–12, 5–25, 18–30 m/s
Gap d	Very Near, Near, Medium, Far	Use calibrated ranges appropriate to scenario.
Brake B	None, Low, Medium, High, Very High	0–100% output universe

Normalisation

For inputs with very different scales, a normalised variable may be used: z = (x – x_min)/(x_max – x_min), clipped to [0,1]. Keep the mapping reversible and document the physical units. Avoid silently extrapolating beyond the domain.

7. Fuzzy Rule Base

The following is an interpretable starting rule base. It is not a validated driving policy. Gap and speed are combined so that a short gap tends to request more braking, especially at high speed.

Gap \ Speed	Slow	Moderate	Fast
Very Near	High	Very High	Very High
Near	Low–Medium	High	Very High
Medium	None–Low	Low–Medium	High
Far	None	None–Low	Low–Medium

Example IF–THEN rules

• R1: IF gap is Very Near AND speed is Fast THEN brake is Very High.

• R2: IF gap is Near AND speed is Moderate THEN brake is High.

• R3: IF gap is Far AND speed is Slow THEN brake is None.

• R4: IF gap is Medium AND speed is Fast THEN brake is High.

• R5: IF gap is Near AND speed is Slow THEN brake is Low–Medium.

Rule completeness checks

• Every valid input pair should activate at least one rule.

• Test values at membership boundaries and at the minimum/maximum input values.

• Look for contradictory outputs, dead zones and sudden command jumps.

• If relative speed is added, expand the rule base and retest all combinations.

Inference example: for AND, a common choice is $\alpha = \min(\mu_{gap}, \mu_{speed})$. A rule's consequent is activated to strength α; aggregation combines all activated consequents. The chosen operators must be stated in code and report.

8. Defuzzification and Command Conditioning

For a discrete output universe y_i with aggregated membership μ_B(y_i), the centroid method is:

B* = Σ[y_i μ_B(y_i)] / Σ[μ_B(y_i)]

If the denominator is zero, the implementation must use a documented fallback (for example, hold the last valid command or request a conservative predefined value). Then clamp the output: B = min(100, max(0, B*)).

Smooth deceleration has two related parts

1. Smooth command: constrain the change in brake request between time steps. 2. Smooth physical response: model acceleration/deceleration and avoid instantaneous velocity changes. These are different; a smooth brake percentage does not alone prove comfortable or safe motion.

Rate limiter formula

Let B_target[k] be the FLC output and r be the maximum allowed command change in percentage points per second:
B[k] = B[k–1] + clip(B_target[k] – B[k–1], –rΔt, +rΔt)

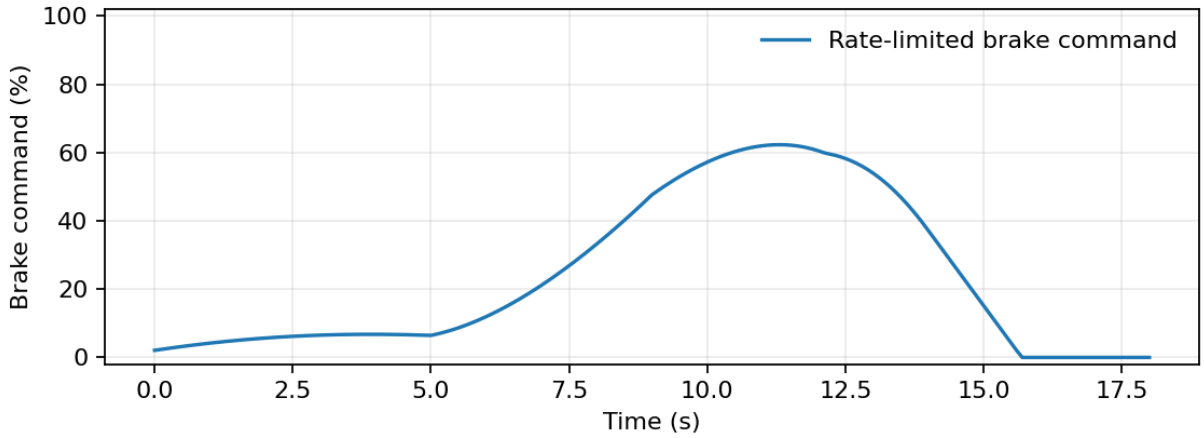


Figure 2. Illustrative rate-limited brake-command trace generated from a synthetic scenario.

Tune r by evaluating response delay, peak braking, stopping distance and command smoothness together. Excessively low r may delay emergency braking; excessively high r may reintroduce abrupt changes.

9. Smooth Deceleration: Worked Numerical Example

Assume a simulated vehicle speed v■ = 12 m/s, time step Δt = 0.5 s, maximum modelled deceleration a_max = 2.5 m/s² and constant brake command B = 40%. Then a = (40/100) × 2.5 = 1.0 m/s².

Step k	Time (s)	Speed before (m/s)	Deceleration (m/s²)	Speed after (m/s)
0	0.0	12.0	1.0	11.5
1	0.5	11.5	1.0	11.0
2	1.0	11.0	1.0	10.5
3	1.5	10.5	1.0	10.0
4	2.0	10.0	1.0	9.5

The discrete update is v[k+1] = max(0, v[k] – aΔt). Under the constant-deceleration idealisation, the continuous-time stopping distance from speed v is approximately d_stop = v²/(2a), and stopping time is t_stop = v/a. For v = 12 m/s and a = 1 m/s², these are about 72 m and 12 s, respectively.

Why this matters

- The example is arithmetic under assumed constant deceleration, not a measured vehicle result.
- Actual braking depends on road friction, slope, tyre condition, mass, actuator lag and control latency.
- If the lead vehicle is also moving, the available gap changes during braking and must be simulated explicitly.
- At v = 0, clamp the speed to zero and record a stopped state; never allow negative velocity.

A safety supervisor may bypass the nominal smoothness limiter in a separately specified critical condition, but this requires careful testing. Do not present a toy simulation as an emergency-braking product.

10. Python Implementation — Core Controller

The following compact example shows triangular membership, a simple interpretable risk-to-brake mapping and a rate limiter. It is a runnable educational baseline; it is not a complete validated fuzzy-inference library implementation.

```
import numpy as np

def tri(x, a, b, c):
    # Handles left/right shoulder sets too.
    if x <= a or x >= c:
        return 1.0 if (x == a and a == b) or (x == c and b == c) else 0.0
    if x <= b:
        return 1.0 if b == a else (x-a)/(b-a)
    return 1.0 if c == b else (c-x)/(c-b)

def clamp(x, lo, hi):
    return max(lo, min(hi, x))

def brake_target(speed_mps, gap_m, lead_speed_mps):
    # Continuous illustrative controller; tune with test scenarios.
    safe_gap = 5.0 + 1.25 * speed_mps
    gap_risk = clamp((safe_gap-gap_m)/max(safe_gap, 1.0), 0, 1)
    closing = max(0.0, speed_mps-lead_speed_mps)
    closing_risk = clamp(closing/12.0, 0, 1)
    return clamp(100*(0.75*gap_risk + 0.25*closing_risk), 0, 100)

def rate_limit(target, previous, rate_pct_per_s, dt):
    delta = clamp(target-previous,
                  -rate_pct_per_s*dt,
                   rate_pct_per_s*dt)
    return clamp(previous+delta, 0, 100)

def update_speed(v, brake_pct, a_max, dt):
    decel = (brake_pct/100.0)*a_max
    return max(0.0, v-decel*dt)
```

Implementation note

The function brake_target above is a continuous risk mapping, not the full membership-rule-centroid pipeline. To implement a strict FLC, calculate input membership degrees, evaluate the IF–THEN rules, aggregate output sets and defuzzify using the centroid formula. The report keeps this distinction explicit so that the code is not mislabelled.

11. Python Simulation Loop and Logging

```
dt = 0.1
duration = 18.0
v_ego = 18.0
x_ego = 0.0
x_lead = 32.0
v_lead = 17.0
a_max = 3.0
brake_prev = 0.0
rate = 22.0                 # percentage points per second
records = []

for k in range(int(duration/dt) + 1):
    t = k * dt
    # Example scenario: lead car slows after 5 s
    if t >= 5.0:
        v_lead = max(9.0, 17.0 - 2.0*(t-5.0))

    gap = max(0.0, x_lead - x_ego - 4.5)
    target = brake_target(v_ego, gap, v_lead)
    brake = rate_limit(target, brake_prev, rate, dt)
    v_next = update_speed(v_ego, brake, a_max, dt)

    records.append((t, v_ego, v_lead, gap, brake))
    x_ego += v_next * dt
    x_lead += v_lead * dt
    v_ego, brake_prev = v_next, brake

# Convert records to a NumPy array or pandas DataFrame.
# Plot time against speed, gap and brake command.
```

Implementation checklist

• Keep the same units in every function and data column.

• Record raw target brake and rate-limited brake separately when tuning.

• Log minimum gap and any gap threshold violation before summarising the run.

• Test a stopped lead vehicle, high initial speed, tiny initial gap and invalid inputs.

• Use a fixed random seed if noise is added, so comparisons are repeatable.

The synthetic graph in this report was generated for visual demonstration. It should not be described as a validated experimental result from a physical car.

12. Visual Results: Speed, Gap and Brake

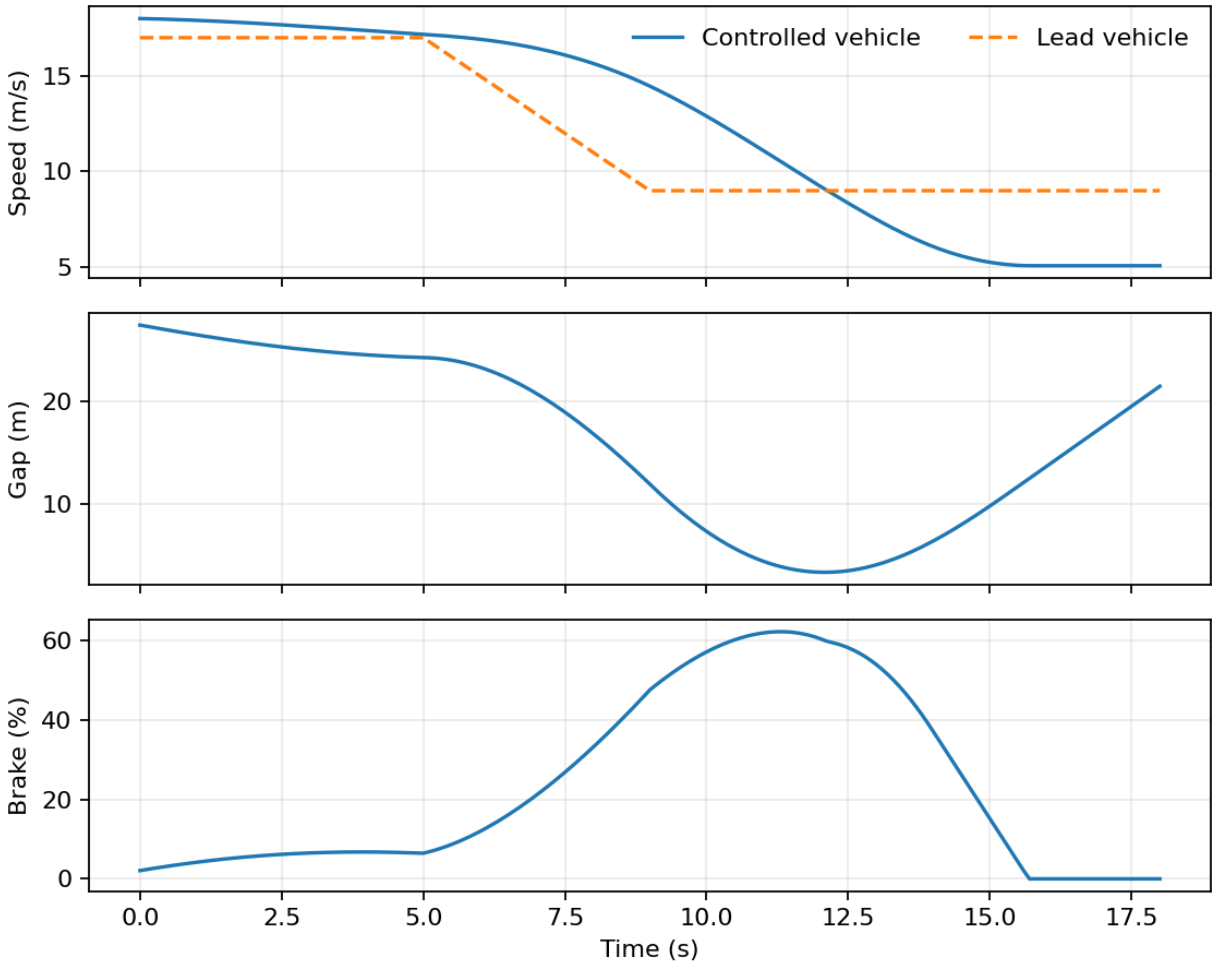


Figure 3. Example time-series visualisation from a synthetic, simplified car-following scenario. The lead vehicle slows after 5 seconds; the controller command is rate-limited. This is an illustrative run, not evidence of real-road safety.

How to read the graphs

• Speed vs time: compare ego speed with lead speed; inspect whether the controlled car responds to the lead vehicle slowing.

• Gap vs time: verify that distance is updated from both vehicle positions; check for gap collapse or negative values.

• Brake vs time: look for abrupt jumps, prolonged saturation, delayed response and unnecessary oscillation.

A proper results section should add numeric values from actual repeated runs: minimum gap, maximum brake command, time to stop, stopping distance, command total variation and gap-violation duration. Do not infer exact performance from a plot alone.

13. Control Surface and Visual Representation

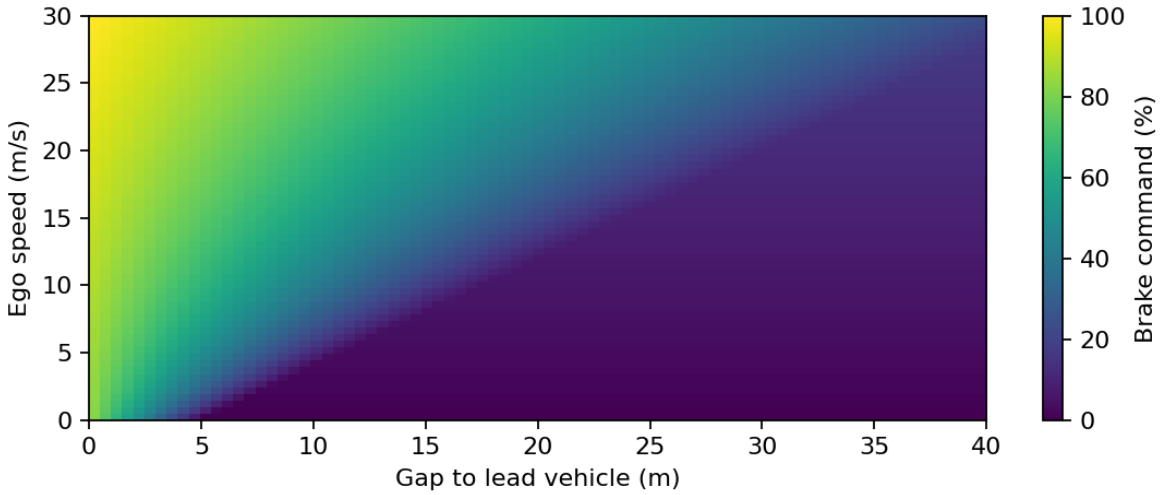


Figure 4. Illustrative two-dimensional control surface for a continuous risk-based command. Higher values indicate stronger braking requests. A true fuzzy controller's surface should be generated by evaluating the actual rule base over a grid of speed and gap inputs.

Visual checks

- Near-zero gap should not map to zero braking at ordinary nonzero speed.
- For the same gap, higher ego speed should generally not reduce braking demand without a reasoned rule.
- For the same speed, a larger gap should usually reduce the braking command.
- Inspect transitions around every membership-function overlap and every rule boundary.
- Test out-of-range values and ensure output remains between 0 and 100%.

For portfolio transparency, label each graph with the controller version, scenario, units, time step and whether data is simulated or measured.

14. Complexity Analysis

Let n be the number of simulation time steps, p the number of input variables, m the maximum number of membership sets per input, R the number of fuzzy rules, and q the number of sampled output points used in centroid defuzzification.

Operation	Per-step cost (typical)	Total for n steps
Input validation and state update	O(1)	O(n)
Membership evaluation	O(pm)	O(npm)
Rule evaluation	O(R)	O(nR)
Sampled aggregation + centroid	O(q) or O(Rq)	O(nq) or O(nRq), depending on implementation
Logging all state variables	O(1) append amortised	O(n) time and O(n) memory
Streaming metrics without full history	O(1) per step	O(n) time and O(1) extra memory

Overall time complexity

For a conventional rule-evaluation implementation with precomputed membership degrees, per-step time is approximately O(pm + R + q) when each rule contributes to a fixed-size output grid efficiently. Total simulation time is O(n(pm + R + q)). If aggregation loops over all R rules for every one of q output samples, the defuzzification component may be O(Rq), yielding O(n(pm + R + Rq)).

Space complexity

If every time step is retained for plotting, memory is O(n). If only current state and running metrics are stored, extra memory can be O(1), excluding fixed controller tables and output arrays. A dense precomputed control surface uses O(N_v N_d) memory for N_v speed points and N_d gap points.

Practical note: fuzzy control with a small fixed rule base is usually inexpensive relative to camera perception. Measure runtime on the target hardware rather than assuming a complexity expression guarantees real-time performance.

15. Evaluation Metrics and Experimental Design

Metric	Definition / calculation	Interpretation
Minimum gap	min_k d[k]	Smallest separation reached in a run.
Gap-violation duration	Σ 1(d[k] < d_min) Δt	Time below a chosen evaluation threshold.
Stopping time	First t where v ≤ ε after braking begins	How long the simulated vehicle takes to stop or settle.
Stopping distance	x_stop – x_brake_start	Distance travelled during braking.
Speed tracking error	e_v[k] = v_ref[k] – v[k]	Deviation from a target-speed profile.
Brake smoothness	Σ |B[k] – B[k–1]|	Total command variation; lower is smoother but may be slower.
Peak deceleration	max a[k]	Largest modelled braking magnitude.
Computation time	wall-clock time / simulated duration	Useful for real-time feasibility checks.

Test matrix

• Scenario A: steady lead speed and a comfortable initial gap.

• Scenario B: lead vehicle slows gradually.

• Scenario C: lead vehicle brakes sharply at a known time.

• Scenario D: initial gap is small and ego speed is high.

• Scenario E: add sensor noise/delay after the ideal baseline works.

Compare FLC and threshold/PID baselines with identical initial conditions, time step, lead-car profile and model parameters. Run each scenario repeatedly when noise is present and report mean, spread and worst case. Set d_min and all thresholds before seeing the results.

16. Baseline Comparison and Ablation Plan

Controller	Decision basis	Strength	Main limitation
Fixed threshold	Gap crosses a fixed boundary	Very easy to implement and explain	Can switch abruptly; little context.
Fuzzy logic	Membership degrees and IF–THEN rules	Interpretable graded decisions	Depends on rule/membership design.
PID (optional)	Proportional, integral and derivative error terms	Well-known feedback baseline	Needs defined error and careful tuning.
FLC + rate limiter	Fuzzy request plus command slew constraint	Can reduce abrupt command changes	Rate limit can delay urgent response.

Ablation experiments

- Run FLC without rate limiting and with rate limiting to isolate smoothness effects.
- Vary the maximum brake-command rate r while measuring both smoothness and minimum gap.
- Vary Δt to check numerical sensitivity.
- Compare two membership-function parameter sets under the same scenario.
- Add relative speed only after the two-input baseline has been tested.

Do not state that fuzzy logic is superior unless the results support it. A fair comparison must use the same physical model and scenario conditions; otherwise differences may come from setup rather than controller design.

17. Edge Cases, Failure Handling and Robustness

Case	Required handling	Why it matters
Gap ≤ 0	Log collision/overlap event; do not hide it by clamping the event away.	Clamping gap to zero is okay for display, but the violation must remain recorded.
Speed = 0	Keep v at zero and record stopped state.	Prevents reverse motion from numerical updates.
No active fuzzy rule	Use documented fallback; emit diagnostic.	Avoids NaN or undefined output.
NaN / infinite input	Reject, sanitize or enter a defined safe fallback.	Invalid data must not propagate.
Brake command saturation	Clamp to [0,100].	Preserves actuator model bounds.
Lead vehicle stops suddenly	Test high-risk scenario and log minimum gap.	A nominal smooth controller may respond too slowly.
Sensor delay/noise	Add explicit delay/noise model and test sensitivity.	Ideal state availability overstates practical capability.

Robustness protocol

• Document the expected input range and behaviour outside it.

• Record every safety-relevant event; never remove failed runs from the summary without explanation.

• Separate a nominal FLC from any emergency override and test the override independently.

• Use conservative, transparent labels: simulated event, threshold violation, or modelled stop—not “accident prevented”.

A rate limiter is not a substitute for an emergency-response design. If immediate braking is necessary, the override policy, activation condition and maximum braking response must be specified and validated separately.

18. Development Roadmap and Portfolio Presentation

Stage	Build task	Evidence to publish
1. Baseline	Implement state update, units and logging.	Source code, requirements, simple test cases.
2. Fuzzy layer	Add membership functions, rules and defuzzification.	Membership plots, rule table, unit tests.
3. Smooth control	Add command rate limiting and stop logic.	Before/after command traces and stopping metrics.
4. Evaluation	Run scenario matrix and controller baselines.	Reproducible graphs, metric table, limitations.
5. Perception extension	Only if needed: camera/object detection plus metric range estimation.	Calibration method, dataset split and range error.
6. Hardware extension	Miniature vehicle with safe test area and emergency stop.	Wiring, measured logs, safety protocol.

Recommended repository structure

```
fuzzy-car-speed-control/
  README.md
  requirements.txt
  src/
    fuzzy_controller.py
    vehicle_model.py
    scenarios.py
    evaluate.py
  tests/
    test_membership.py
    test_bounds.py
    test_edge_cases.py
  results/
    speed_gap_brake.png
    control_surface.png
    metrics.csv
  docs/
    technical_report.pdf
```

README should state: project goal, scope, how to install/run, parameter table, sample graphs, test scenarios, measured or simulated status, known limitations and licence. Include a concise GIF only if it clearly represents the simulation; never imply a simulation is a real vehicle.

19. Conclusion, Limitations and References

This report specifies a practical path for building a fuzzy-logic speed-control simulation: define a consistent vehicle model, design overlapping membership functions, cover the input space with readable rules, defuzzify the result, condition the command, and evaluate repeatable scenarios using graphs and metrics. Smooth deceleration is treated as a control-design issue rather than a visual claim alone.

Current limitations

• The plotted traces are synthetic demonstrations generated from a simplified model.

• The core example uses virtual state values; real camera-based metric distance is not implemented.

• The linear brake-to-deceleration model omits tyre-road friction, slope, actuator delay and vehicle-specific dynamics.

• No result here certifies safety, road readiness or superiority over another controller.

Further work should implement and test the full fuzzy inference pipeline, compare it against baselines, introduce relative speed and sensor uncertainty, and report results from repeated, reproducible experiments. Hardware testing should only follow with appropriate safety precautions.

References and source basis

1. User-provided file: *Combined_Fuzzy_Logic_Car_Speed_Control_MTech_Report.pdf* — used for project scope, architecture, model assumptions, rule table, evaluation criteria and limitations.

2. User-provided file: *IMPLEMENTATION IDEA.pdf* — handwritten implementation notes covering the simulation workflow, braking equation, example deceleration table, plotting and evaluation checklist.

3. AlGhamdi, A. S. and Eren, H., “Using FUZZY Logic to Control Vehicles’ Velocity in Traffic System,” IFAC Intelligent Autonomous Vehicles Symposium, 2013, pp. 54–59. Used as background context for fuzzy vehicle-speed control; this report does not reproduce its full method.

4. The equations, parameter values, synthetic graphs and Python snippets in this report are educational design examples. They must be tested and tuned before being described as experimental findings.

END OF TECHNICAL REPORT